

Exercícios de Fixação – SQL no MySQL

T06 – Banco de Dados | Prof. Calvetti

Nome: Eduardo Rafael de Moraes Juvenasso.

RA:825135809

Exercício 1

1. Criar o schema e selecionar:

```
CREATE DATABASE universidade;  
USE universidade;
```

2. Criar as tabelas na ordem das dependências: primeiro ALUNO e DISCIPLINA, depois MATRICULA.

```
CREATE TABLE aluno (  
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(120) NOT NULL,  
    email VARCHAR(254) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE disciplina (  
    id_disciplina INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(120) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE matricula (  
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,  
    id_aluno INT NOT NULL,  
    id_disciplina INT NOT NULL,  
    situacao VARCHAR(15) NOT NULL DEFAULT 'ATIVA',  
    UNIQUE (id_aluno, id_disciplina),  
    CHECK (situacao IN ('ATIVA','EM_CURSO','CONCLUIDA','CANCELADA')),  
    FOREIGN KEY (id_aluno) REFERENCES aluno(id_aluno),  
    FOREIGN KEY (id_disciplina) REFERENCES disciplina(id_disciplina)  
) ENGINE=InnoDB;
```

3. Adicionar carga_horaria em DISCIPLINA com ALTER TABLE:

```
ALTER TABLE disciplina ADD COLUMN carga_horaria INT NOT NULL DEFAULT 40 CHECK (carga_horaria > 0);
```

4. Conferir as tabelas e a estrutura final:

```
SHOW TABLES;  
SHOW CREATE TABLE disciplina;  
SHOW CREATE TABLE matricula;
```

5. A ordem usada é ALUNO e DISCIPLINA → MATRICULA, porque MATRICULA depende das duas FKs.

Exercício 2

1. Inserir três alunos e três disciplinas de uma vez:

```
INSERT INTO aluno (nome,email) VALUES  
('Ana Lima','ana@email.com'),  
('Bruno Silva','bruno@email.com'),  
('Carlos Souza','carlos@email.com');
```

```
INSERT INTO disciplina (nome,carga_horaria) VALUES  
('Banco de Dados',60),
```

```
('Programação',80),  
( 'Redes',40);
```

2. Consultar as chaves geradas:

```
SELECT id_aluno,nome FROM aluno;  
SELECT id_disciplina,nome FROM disciplina;
```

3. Inserir matrículas usando IDs que realmente existem:

```
INSERT INTO matricula (id_aluno,id_disciplina)  
VALUES  
(1,1),  
(2,2),  
(3,3);
```

4. Provar uma matrícula com aluno inexistente:

```
INSERT INTO matricula (id_aluno,id_disciplina) VALUES (999,1);
```

5. O erro esperado é de FOREIGN KEY, porque o aluno 999 não existe. A correção é usar um id_aluno que exista, sem desabilitar a FK.

Exercício 3

1. Ver as matrículas ATIVAS de 2026 antes de alterar:

```
SELECT *  
FROM matricula  
WHERE situacao = 'ATIVA';
```

2. Atualizar somente esse conjunto:

```
UPDATE matricula  
SET situacao = 'EM_CURSO'  
WHERE situacao = 'ATIVA';
```

3. Tentar excluir uma disciplina que possui matrículas:

```
DELETE FROM disciplina WHERE id_disciplina = 1;
```

4. Se houver matrícula ligada à disciplina, a FK com RESTRICT impede a exclusão. Isso evita apagar uma disciplina que ainda está sendo usada.

5. Sem WHERE, o UPDATE ou DELETE poderia atingir todas as linhas da tabela. Por isso é importante fazer primeiro o SELECT com o mesmo WHERE.

Desafio Final

1. Criar o schema da clínica e as cinco tabelas:

```
CREATE DATABASE clinica;  
USE clinica;
```

```
CREATE TABLE paciente (  
  id_paciente INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  cpf CHAR(11) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE medico (  
  id_medico INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  crm VARCHAR(20) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE consulta (
  id_consulta INT AUTO_INCREMENT PRIMARY KEY,
  id_paciente INT NOT NULL,
  id_medico INT NOT NULL,
  data_hora DATETIME NOT NULL,
  estado VARCHAR(15) NOT NULL DEFAULT 'AGENDADA',
  UNIQUE (id_medico,data_hora),
  CHECK (estado IN ('AGENDADA','REALIZADA','CANCELADA')),
  FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente),
  FOREIGN KEY (id_medico) REFERENCES medico(id_medico)
) ENGINE=InnoDB;
```

```
CREATE TABLE procedimento (
  id_procedimento INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(120) NOT NULL UNIQUE,
  valor DECIMAL(10,2) NOT NULL CHECK (valor >= 0)
) ENGINE=InnoDB;
```

```
CREATE TABLE consulta_procedimento (
  id_consulta INT NOT NULL,
  id_procedimento INT NOT NULL,
  quantidade INT NOT NULL DEFAULT 1 CHECK (quantidade > 0),
  valor_praticado DECIMAL(10,2) NOT NULL CHECK (valor_praticado >= 0),
  PRIMARY KEY (id_consulta,id_procedimento),
  FOREIGN KEY (id_consulta) REFERENCES consulta(id_consulta) ON DELETE CASCADE,
  FOREIGN KEY (id_procedimento) REFERENCES procedimento(id_procedimento) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

2. Fazer duas alterações no schema:

```
ALTER TABLE paciente ADD COLUMN data_nascimento DATE NULL;
ALTER TABLE medico ADD COLUMN ativo BOOLEAN NOT NULL DEFAULT TRUE;
```

3. Povoar primeiro os pais e depois os filhos:

```
INSERT INTO paciente (nome,cpf) VALUES
('Ana Lima','11111111111'),
('Bruno Silva','22222222222');
```

```
INSERT INTO medico (nome,crm) VALUES
('Dr. Carlos','CRM100'),
('Dra. Fernanda','CRM200');
```

```
INSERT INTO procedimento (nome,valor) VALUES
('Consulta','150.00'),
('Exame','80.00');
```

```
INSERT INTO consulta (id_paciente,id_medico,data_hora)
VALUES (1,1,'2026-09-30 10:00:00');
```

```
INSERT INTO consulta_procedimento
(id_consulta,id_procedimento,quantidade,valor_praticado)
VALUES (1,1,1,150.00);
```

4. Antes de UPDATE e DELETE, fazer o SELECT equivalente:

```
SELECT * FROM consulta WHERE id_consulta = 1;
UPDATE consulta SET estado = 'REALIZADA'
WHERE id_consulta = 1;
```

```
SELECT * FROM procedimento WHERE id_procedimento = 2;
DELETE FROM procedimento WHERE id_procedimento = 2;
```

5. Quatro erros de integridade que podem ser testados:

```
-- CPF repetido
INSERT INTO paciente (nome,cpf)
VALUES ('Outro','11111111111');

-- paciente inexistente
INSERT INTO consulta (id_paciente,id_medico,data_hora)
VALUES (999,1,'2026-10-01 10:00:00');

-- valor negativo
INSERT INTO procedimento (nome,valor)
VALUES ('Teste',-10);

-- médico inexistente
INSERT INTO consulta (id_paciente,id_medico,data_hora)
VALUES (1,999,'2026-10-01 11:00:00');
```

6. Os erros esperados são UNIQUE para CPF repetido, FOREIGN KEY para IDs inexistentes e CHECK para valor negativo.

7. Finalizar conferindo as estruturas:

```
SHOW CREATE TABLE paciente;
SHOW CREATE TABLE medico;
SHOW CREATE TABLE consulta;
SHOW CREATE TABLE procedimento;
SHOW CREATE TABLE consulta_procedimento;

SHOW INDEX FROM consulta;
SHOW INDEX FROM consulta_procedimento;
```